```
import pandas as pd
import re
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


# Load dataset
df = pd.read_csv("mtsamples.csv")

# Remove rows with missing values
df = df.dropna(subset=["transcription", "medical_specialty"])

# Keep only common specialties to make classification manageable
top_classes = df["medical_specialty"].value_counts().head(5).index
df = df[df["medical_specialty"].isin(top_classes)]

# Text preprocessing
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

df["clean_text"] = df["transcription"].apply(clean_text)

# Input and output
X = df["clean_text"]
y = df["medical_specialty"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# TF-IDF feature extraction
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=5000
)
```

```
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# SVM classifier
model = LinearSVC()
model.fit(X_train_tfidf, y_train)

# Prediction
y_pred = model.predict(X_test_tfidf)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Total samples:", len(df))
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Feature size:", X_train_tfidf.shape[1])

print("\nAccuracy:", round(accuracy * 100, 2), "%")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=model.classes_
)

disp.plot(xticks_rotation=45)
plt.title("Clinical Text Classification - Confusion Matrix")
plt.tight_layout()
plt.show()
```